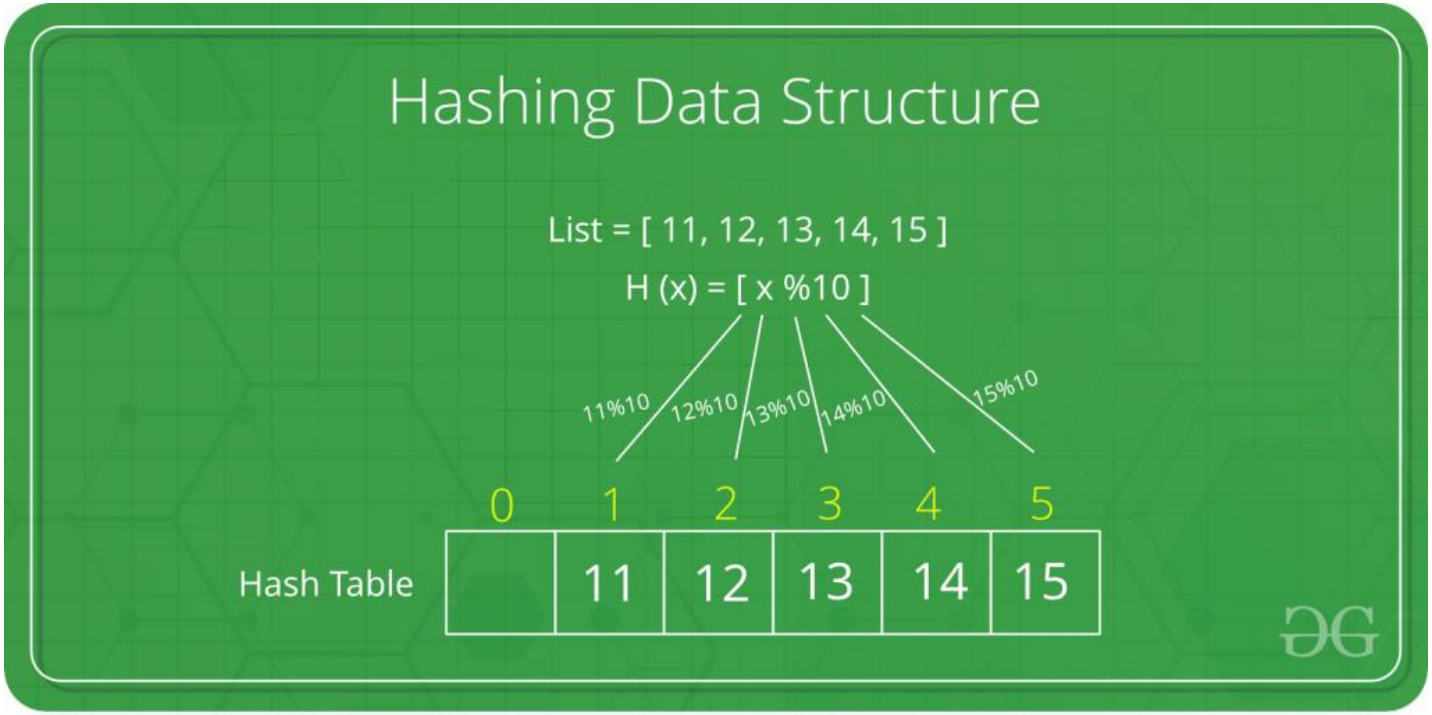
Hashing

Why?

- Many applications deal with lots of data (Search engines and web pages)
- The look ups are time critical.
- Typical data structures like arrays and lists, may not be sufficient to handle efficient lookups
- In general: When look-ups need to occur in near constant time. O(1)

- Hashing is Data Structure which is designed to use a special function called the Hash function
- It is used to map a given value with a particular key for faster access of elements.
- The efficiency of mapping depends of the efficiency of the hash function used.
- Appli- Web search, database, compiler, passwords

- hash function H(x) maps the value at the index **x%10** in an Array



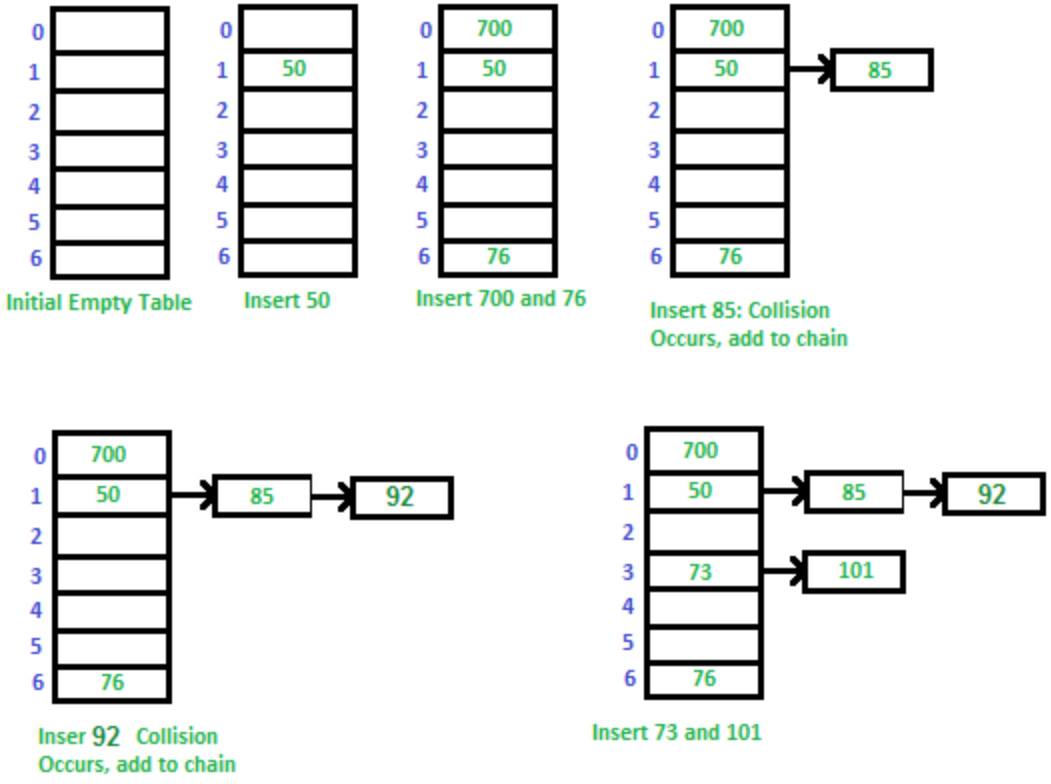
- **Hash Function:** A function that converts a given big phone number to a small practical integer value.
- The mapped integer value is used as an index in hash table.
- In simple terms, a hash function maps a big number or string to a small integer that can be used as index in hash table.
  A good hash function should have following properties
  1) Efficiently computable.
  2) Should uniformly distribute the keys (Each table position equally likely for each key)

Hash table

- **Hash table-** Array that stores pointers to records corresponding to a given phone number.
- **Collision Handling-**Hash function gets us a small number for a big key, there is possibility that two keys result in same value. Newly inserted key maps to an already occupied slot in hash table is called collision

- **Chaining:**
  - The idea is to make each cell of hash table point to a *linked list* of records that have same hash function value.
  - Chaining is simple, but requires additional memory outside the table.
- **Open Addressing:**
  - In open addressing, all elements are stored in the hash table itself.
  - Each table entry contains either a record or NIL.
  - When searching for an element, we one by one examine table slots until the desired element is found or it is clear that the element is not in the table.

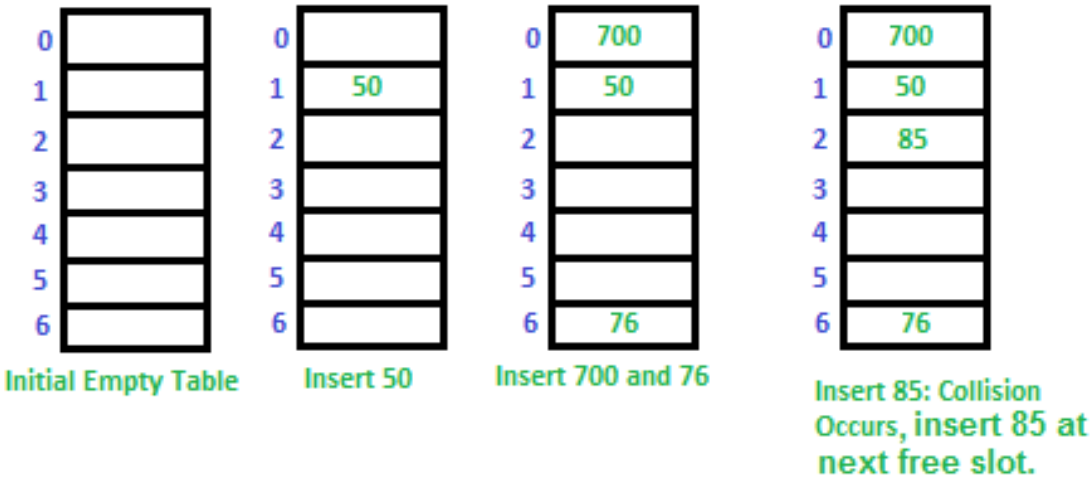
- “**key mod 7**” Keys- 50, 700, 76, 85, 92, 73, 101.

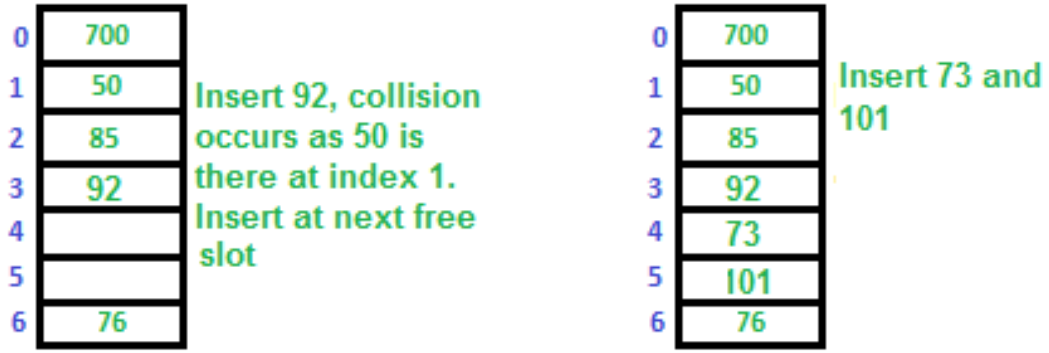


- **Advantages:**
  1) Simple to implement.
  2) Hash table never fills up, we can always add more elements to the chain.
  3) Less sensitive to the hash function or load factors.
  4) It is mostly used when it is unknown how many and how frequently keys may be inserted or deleted.
- **Disadvantages:**
  1) Cache performance of chaining is not good as keys are stored using a linked list. Open addressing provides better cache performance as everything is stored in the same table.
  2) Wastage of Space (Some Parts of hash table are never used)
  3) If the chain becomes long, then search time can become O(n) in the worst case.
  4) Uses extra space for links.

- m = Number of slots in hash table
- n = Number of keys to be inserted in hash table
  Load factor
- $\alpha = n/m$
- Expected time to search = $O(1 + \alpha)$
- Expected time to delete = $O(1 + \alpha)$
- Time to insert = $O(1)$
- Time complexity of search insert and delete is $O(1)$ if α is $O(1)$

- “key mod 7”, keys - 50, 700, 76, 85, 92, 73, 101.





- **Insert(k):** Keep probing until an empty slot is found. Once an empty slot is found, insert k.
- **Search(k):** Keep probing until slot's key doesn't become equal to k or an empty slot is reached.
- **Delete(k):** If we simply delete a key, then the search may fail. So slots of deleted keys are marked specially as "deleted".

- ***Linear Probing:*** In linear probing, we linearly probe for next slot.
  - If slot hash(x) % S is full, then we try (hash(x) + 1) % S
  - If (hash(x) + 1) % S is full, then we try (hash(x) + 2) % S

Challenges in Linear Probing :

- **Primary Clustering:** One of the problems with linear probing is Primary clustering, many consecutive elements form groups and it starts taking time to find a free slot or to search an element.
- **Secondary Clustering*:*** Secondary clustering is less severe, two records do only have the same collision chain(Probe Sequence) if their initial position is the same.

Quadratic Probing We look for i^2‘th slot in i’th iteration.

let hash(x) be the slot index computed using hash function.

- If slot hash(x) % S is full, then we try (hash(x) + 1*1) % S
- If (hash(x) + 1*1) % S is also full, then we try (hash(x) + 2*2) % S
- If (hash(x) + 2*2) % S is also full, then we try (hash(x) + 3*3) % S

- **Double Hashing** We use another hash function hash2(x) and look for i*hash2(x) slot in i'th rotation.

let hash(x) be the slot index computed using hash function.

- If slot hash(x) % S is full, then we try (hash(x) + 1*hash2(x)) % S
- If (hash(x) + 1*hash2(x)) % S is also full, then we try (hash(x) + 2*hash2(x)) % S

Comparison of above three:

- Linear probing has the best cache performance but suffers from clustering. One more advantage of Linear probing is easy to compute.
- Quadratic probing lies between the two in terms of cache performance and clustering.
- Double hashing has poor cache performance but no clustering. Double hashing requires more computation time as two hash functions need to be computed.

- **Multiple-choice hashing**: Give each element multiple choices for positions where it can reside in the hash table
- **Relocation hashing**: Allow elements in the hash table to move after being placed

Cuckoo Hashing

- Cuckoo hashing applies the idea of multiple-choice and relocation together and guarantees O(1) worst case lookup time!
- **Multiple-choice:** We give a key **two choices** h1(key) and h2(key) for residing.
- **Relocation**: It may happen that h1(key) and h2(key) are preoccupied. Inserting a new key into a cuckoo hashing table may push an older key to a different location. This leaves us with the problem of re-placing the older key.
  - If alternate position of older key is vacant, there is no problem.
  - Otherwise, older key displaces another key. This continues until the procedure finds a vacant position, or enters a cycle. In case of cycle, new hash functions are chosen and the whole data structure is 'rehashed'. Multiple rehashes might be necessary before Cuckoo succeeds.

- {20, 50, 53, 75, 100, 67, 105, 3, 36, 39}
- h1(key) = key%11
- h2(key) = (key/11)%11

	20	50	53	75	100	67	105	3	36	39
h_1(key)	9	6	9	9	1	1	6	3	3	6
h_2(key)	1	4	4	6	9	6	9	0	3	3